

01

研究背景与意义

Background & Motivation

从 CVE 增长趋势出发，看清编译器防御机制为何成为脆弱的关键环节

研究背景与意义 — 漏洞过载时代

近十年 CVE 数量持续攀升，编译器侧防御机制成为系统安全的关键防线

图 1: 近 10 年 CVE 年度发布数



编译器侧防御：系统安全的关键防线

- 栈金丝雀 · `_FORTIFY_SOURCE`
- CFI · IBT · PAC · BTI
- Shadow Stack · SafeStack

由编译器与运行时叠加部署，在漏洞被触发时拦截攻击者拿到控制权。

核心命题

上层代码的内存安全漏洞难以根除。防御机制能否在漏洞被触发时挡住攻击者，直接决定整套系统是否守得住。

研究背景与意义 — 防御机制本身也会出错

防御机制由代码实现，其有效性与 ISA 设计强相关 — Canary 在 AArch64 上的失效



Canary 原理

编译器在缓冲区与返回地址之间放置随机值哨兵。

函数返回前比对哨兵是否被改写，一旦改写就调用 `__stack_chk_fail` 终止程序。

AArch64 上失效的根因 (CVE-2023-4039)

GCC 把 canary 固定放在**局部变量区的最上方**。

但 AArch64 栈帧把 VLA / alloca 放在**保存的返回地址之下**，越界写入先于 canary 检查改写返回地址，哨兵被绕开。

图 2: AArch64 栈帧布局 (CVE-2023-4039)

缺陷自 GCC 早期版本起存在，编译时无报错、运行时功能正常，直到 2023 年由外部研究者发现并修复。

研究背景与意义 — 核心挑战

机制 × ISA 二维矩阵庞大，失效又难以判定

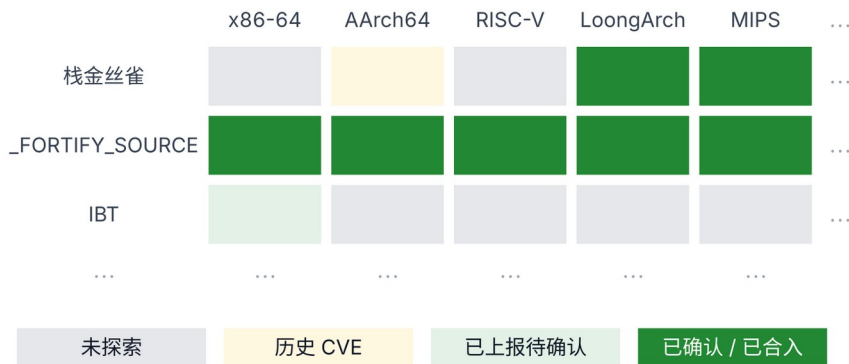


图 3: 防御机制 × ISA 矩阵示意

C1 • 搜索空间巨大

防御机制十余种 (canary · FORTIFY · CFI · IBT · PAC · BTI · ShadowStack ...), ISA 至少 4 套 (x86-64 · AArch64 · RISC-V · LoongArch)。每个格点都对应一份独立的后端实现，人工审计难以覆盖。

C2 • 失效难以判定 (Oracle 问题)

程序不崩溃、跨编译器输出一致，看似正常而安全契约已失守。传统 fuzzer 依赖 crash / differential oracle，对这类静默逻辑失效无感。

本课题的应对思路

以 invariant 驱动的预言机回答 Oracle 问题，结合覆盖率反馈与 LLM 种子生成压缩搜索空间（详见后章）。

02

研究现状

Related Work

静态分析与现有 fuzz 工作为何都看不见防御机制的静默失效

研究现状 — 现有方法的局限

现有自动化方法均不能直接判定防御机制是否仍生效

静态分析与形式化

依赖已知的代码模式或可形式化的契约。

但防御机制失效形态多样，大部分案例都有独特的违例方式；

安全契约本身又散落在中端 pass 与多个后端模板中，难以统一形式化。

WhiteFox · OSDI'24 · LLM 阅读优化 pass

白盒思路：LLM 从优化 pass 源码归纳输入模式，再生成可触发该 pass 的 C 程序。目标仍是常规 crash / miscompile，不关心安全契约。

HLPFuzz · USENIX Sec'25 · LLM 约束求解 + 覆盖率

把未覆盖基本块翻译为约束求解任务交给 LLM，渐进式构造上下文。目标是可达性，预言机仍是 crash / 差分输出。

Oracle 空白

程序不崩溃、跨编译器输出一致 → 防御机制是否仍满足安全契约，现有 crash / differential / 变形 oracle 全部沉默。

03

研究目标与内容

Goals & Content

目标：找出 CVE-2023-4039 这类静默失效；内容：不变量 → 预言机 → LLM 种子

研究目标与内容 — 研究目标

找出 GCC / LLVM 中 CVE-2023-4039 这类静默失效，输出可被上游确认的真实缺陷

目标参照：CVE-2023-4039 · AArch64 上 -fstack-protector 形同虚设，编译时无错、运行时正常，但 canary 已被绕开

本课题以「防御机制是否仍满足安全契约」为新判据，替代传统的 crash / 差分预言机，在「机制 × ISA」矩阵上系统性地暴露此类「双重无声」失效。

G (a)

安全不变量集合

- 覆盖主流防御机制 (canary · FORTIFY · IBT 等)
- 每条不变量给出可机器判定的证据形式
- GCC 与 LLVM 共用同一描述结构

G (b)

Fuzzer 原型

- 同时面向 GCC 与 LLVM
- 跨 ISA: x86-64 · AArch64 · RISC-V · LoongArch
- 防御机制相关代码覆盖率为推进信号
- 由 LLM 完成精细变异

G (c)

真实缺陷验证

- 发现 CVE-2023-4039 类的静默失效
- 推动上游确认与回归测试合入
- 验证方法的跨编译器迁移性

研究目标与内容 — 研究内容

三项工作：不变量调研 → 预言机工程化 → LLM × 覆盖率联合驱动

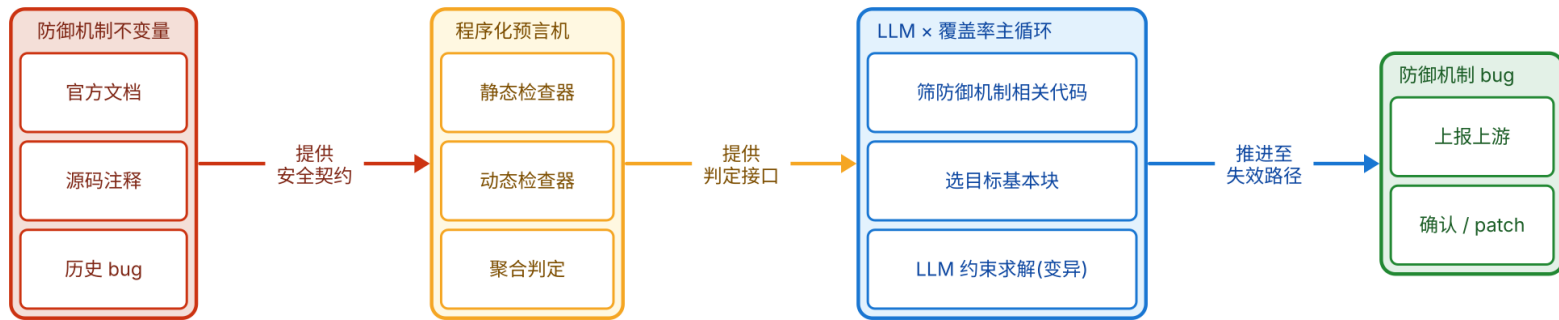


图 4: 研究内容总览 — 不变量为基础, 预言机给判据, LLM × 覆盖率把测试推进到机制代码路径

① 安全不变量调研与形式化

- 三类来源：源码注释 · 官方文档 · 历史 bug
- 统一字段：陈述 · 证据 · 版本敏感性

② 不变量到预言机的工程化

- 静态：ELF 符号 · 指令字节 · .note
- 动态：受控执行 · 二分搜索
- 任一 Fail 即记机制级违例 + 开 / 关对照

③ LLM × 覆盖率主循环

- 按「未覆盖 × 机制相关性」挑目标基本块
- 机制契约 + 目标上下文 → LLM 生成种子
- 新覆盖但未命中：定位发散点回喂模型

04

研究方案

Technical Route

总体架构 → 预言机框架 → LLM × 覆盖率主循环 × 跨架构调度

研究方案 — 总体架构

Fuzz 引擎为骨干, LLM × 编译器 × 预言机三子系统协同

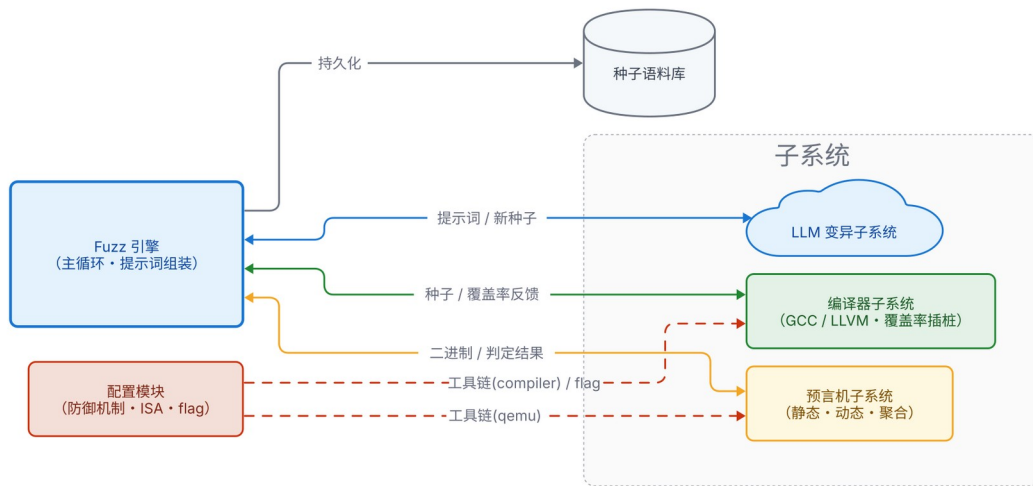


图 5: 总体架构 — 子系统分工

- Fuzz 引擎: 主循环调度 + 提示词组装
- 编译器子系统: 封装插桩 GCC / LLVM, 统一「行级覆盖 → 种子 ID」映射
- 预言机子系统: 按机制聚合检查器, 输出安全判定; 配置模块按「机制 · ISA · flag」三元组下发任务

研究方案 — 预言机框架

静态 + 动态检查器，聚合后回答「防御是否仍有效」

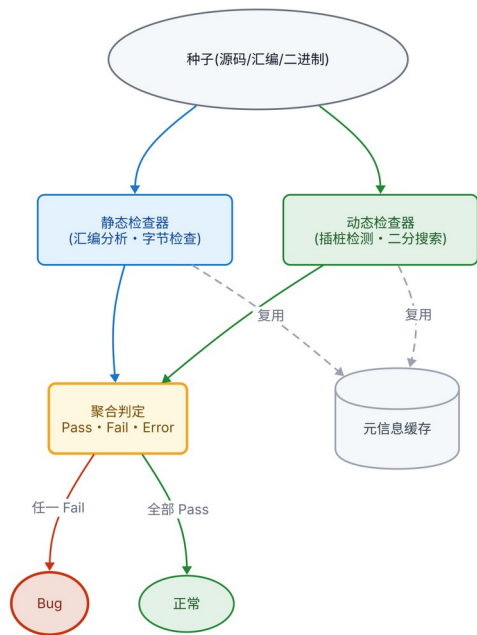


图 6: 预言机判定流程

静态检查器

- ELF 符号表断言（如 `__stack_chk_fail` 是否被引用）
- 指令字节模式扫描（如 `endbr` 出现位置）
- .note 段属性核对（CET / BTI 标志位）

动态检查器

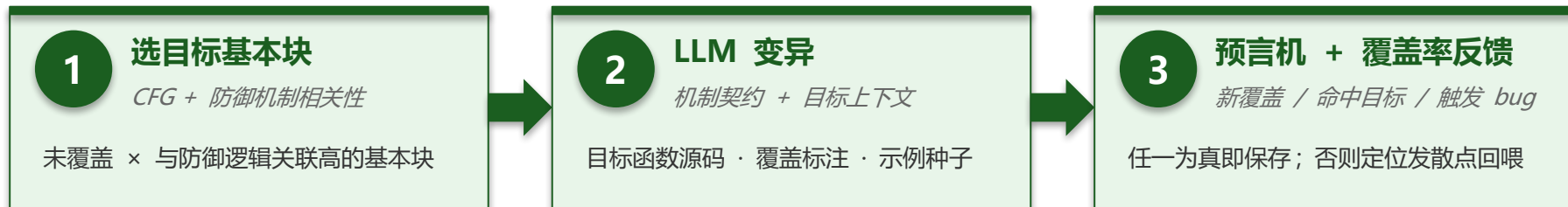
- 受控执行 + 二分搜索，观察越界写入是否被拦截
- 内联汇编探针，检测寄存器残留（canary 泄漏判据）
- QEMU-user 跨架构跑同一颗种子做对照

聚合判定与对照实验

检查器返回 Pass / Fail / Error; 任一 Fail 即记机制级违例并附证据; 同时跑开 / 关机两套 flag 做对照, 避免与编译器缺省行为混淆。

研究方案 — LLM × 覆盖率主循环 + 跨架构调度

单轮迭代：选目标 → LLM 变异 → 预言机 / 覆盖率反馈；同一种子多后端对照



跨架构与编译选项调度

- 配置模块按「机制 · ISA · flag」三元组下发任务；同一颗种子分发至 x86-64 / AArch64 / RISC-V / LoongArch 后端经 QEMU 执行。
- 开 / 关机制双 flag profile 对照，把检查器输出与编译器缺省行为区分开。

05

现有成果

Preliminary Results

已在 GCC 多 ISA × 多版本上发现 3 例已上报缺陷, 其中 2 例已合入 master

现有成果 — GCC 上的三例静默失效

已在 GCC 多版本 × 多 ISA 上发现 3 例已上报缺陷, 其中 2 例已合入 master

Bug 1

`__FORTIFY_SOURCE` size 静默放宽

位置: tree-ssa-strlen pass

上游: PR-125079

strlen pass 把 `__strcat_chk` 改写为 `__strcpy_chk` 时, 第三个 `size` 参数没有按 `strlen(dst)` 减小, 运行时上限被悄悄放宽。

✓ 已合入 master

Bug 2

IBT endbr 字节模式漏检

位置: x86 后端 endbr 谓词

上游: PR-125084

扫描循环在「立即数高位非零」时提前退出, `endbr64` 字节排布在高位即可绕过检测, `.text` 中混入伪 landing pad。

⌚ 等待维护者回复

Bug 3

多 ISA canary 寄存器残留

位置: 通用 SSP fallback 后端

上游: PR-125045 (meta) • PR-125049

MIPS / LoongArch 等 ISA 的 fallback 后端在 epilogue 未清零 canary 寄存器, `__stack_chk_guard` 在返回时残留。

✓ LoongArch 子例已合入

实验范围: GCC × {aarch64-v12.2 · aarch64-v15.2 · x86_64-v15.2 · loongarch64-v15.2}; LLVM 接入与其他防御机制 (SafeStack / PAC / BTI / Shadow Stack) 在下一阶段展开。

现有成果 — Canary 寄存器残留：原理与历史教训

Canary 值不得泄漏 — 因此存储 canary 的寄存器必须在函数返回前清零

Canary 寄存器为何必须清零

- ① Canary 值来自 `__stack_chk_guard`, 同一进程或线程内多次复用。一旦攻击者拿到这个值, 后续金丝雀检查全部形同失效。
- ② Canary 必须频繁进入寄存器: 函数序言中加载、对照前比较、写入到栈帧。这些操作都会让 canary 值短暂出现在通用寄存器里。
- ③ 因此安全规范要求: **存储过 canary 的寄存器, 函数返回前必须显式清零**; 否则 canary 值会随返回值泄漏给调用者乃至攻击者。

历史教训: ARM 已经踩过这个坑

2020 年 **PR-96191** 修复了 ARM / AArch64 的 canary 寄存器残留, 在 epilogue 中显式清零承载 canary 的寄存器。

但 GCC 的通用 SSP fallback 后端并未被同步修复, MIPS / LoongArch / xtensa / csky 等 ISA 走的是 fallback 路径。

这条同源遗漏在 5 年内无人察觉 — 因为程序不崩溃、二进制看起来仍带 canary, 但 **guard 值随时可能从寄存器漏出来**。

我们的做法: 把「epilogue 必须 scrub canary 寄存器」这条规则形式化为一条动态不变量, 作为预言机检查器的判据 — 详见下页。

现有成果 — 用内联汇编 checker 触发多 ISA 泄漏

动态 checker 发现 LoongArch / MIPS 仍存在泄漏; LoongArch 维护者认定为严重漏洞

动态 checker 设计

在受测函数返回前插入内联汇编探针, 把候选寄存器的当前值与 `__stack_chk_guard` 比较; 命中即判定为 **Fail (canary 泄漏)**。

LoongArch 探针示例 (PR-125049)

```
asm volatile (  
    "la.local $t1, __stack_chk_guard\n"  
    "ld.d    $t1, $t1, 0\n"  
    "beq     $t1, $r12, leaked\n"  
); // 返回前若 $r12 还存着 guard 值 → 泄漏
```

Re: [PATCH] LoongArch: harden SSP canary set and test routines [PR 125049]



Xi Ruoyao 2026-05-08 17:14

收件人: Lulu Cheng、gcc-patches | 抄 送: WANG Xuerui、Mingcong Bai、Zixing Liu、我 [详情](#)

On Thu, 2026-05-07 at 10:13 +0800, Lulu Cheng wrote:

> LGTM for trunk.

>

> I consider this a **severe security vulnerability** for LoongArch, so we
> should backport it.

>

> What do you think?

I agree. I'll push it to trunk and backport if no complains in a few days.

图 7: LoongArch 维护者邮件 (PR-125049 patch review)

上游反馈

LoongArch 维护者 Xi Ruoyao 将其确认为 **"severe security vulnerability"**, 决定把修复 backport 至旧版本 GCC; meta-bug PR-125045 跟踪的其他 ISA 子例正在依次上报。